

Human Development Index (HDI) Predictor: A Comprehensive Measure of Well-Being

Project Description:

The **Human Development Index (HDI) Predictor** is a machine learning-based web application that predicts a country's Human Development Index (HDI) score using key socioeconomic indicators. The application provides a simple and interactive interface where users can enter **Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita** to obtain an instant HDI prediction and development category.

Built using **Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn**, the system uses a **Linear Regression** model trained on the Human Development Index dataset. The application performs data preprocessing, model prediction, and result visualization through a user-friendly web interface. It helps students, researchers, policymakers, and development analysts analyze human development efficiently without requiring advanced machine learning knowledge.

Scenarios

Scenario 1: Predicting Very High Human Development

A policymaker enters high values for **Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita**. The system predicts a **Very High HDI** score and classifies the country accordingly, helping evaluate national development performance.

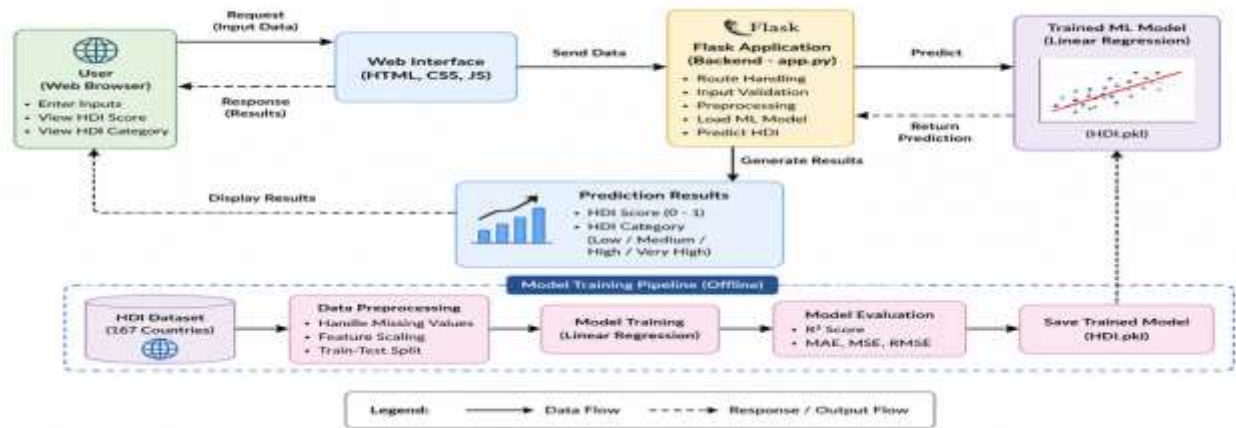
Scenario 2: Assessing Developing Countries

A researcher inputs moderate socioeconomic indicators representing a developing country. The application predicts a **Medium HDI** score and highlights the country's current development status for comparative analysis.

Scenario 3: Identifying Development Challenges

A student enters relatively low values for health, education, and income indicators. The model predicts a **Low HDI** score, helping users understand the factors contributing to lower human development.

Technical Architecture:



Description:

The application follows a **three-layer architecture**:

- **Frontend:** HTML5, CSS3, and Jinja2 templates provide an interactive web interface.
- **Backend:** Flask processes user inputs, validates data, loads the trained machine learning model, and generates predictions.
- **Machine Learning Layer:** A **Scikit-learn Linear Regression** model predicts the HDI score and classifies it into **Low, Medium, High, or Very High** development categories.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Jupyter Notebook**
- **Matplotlib & Seaborn**

Project Workflow:

Milestone 1: Environment Setup & Dataset Preparation

- Install Python libraries and create the virtual environment.
- Download and understand the HDI dataset.
- Perform data preprocessing and exploratory data analysis.

Milestone 2: Machine Learning Model Development

- Train the Linear Regression model.
- Evaluate model performance.
- Save the trained model using Pickle.

Milestone 3: Flask Application Development

- Develop the Flask backend.
- Implement prediction logic.
- Create REST API endpoints.

Milestone 4: Frontend Development

- Design responsive web pages using HTML, CSS, and Jinja2.
- Display prediction results and HDI category.

Milestone 5: Testing & Deployment

- Test the application with different input scenarios.
- Deploy and run the application using the Flask development server.

Milestone 6: Documentation & Conclusion

- Prepare project documentation, README, and user guide.
- Summarize project outcomes and future enhancements.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting an appropriate machine learning algorithm for predicting the **Human Development Index (HDI)** based on socioeconomic indicators. The project also defines the overall system architecture, including the frontend, backend, machine learning model, and data processing components required for accurate HDI prediction.

Activity 1.1: Dataset Collection and Preparation

- **Step 1 — Download the Dataset:** Download the **Human Development Index (HDI) Dataset** from Kaggle.
- **Step 2 — Understand the Dataset:** Examine the dataset containing features such as Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, Gross National Income (GNI) per Capita, and HDI Score.
- **Step 3 — Load the Dataset:** Import the dataset into a Pandas DataFrame using Python.
- **Step 4 — Explore the Dataset:** Inspect the dataset structure, identify missing values, check data types, and generate descriptive statistics.
- **Step 5 — Data Preprocessing:** Handle missing values using mean imputation, select relevant features, and prepare the dataset for machine learning.
- **Step 6 — Split the Dataset:** Divide the data into training and testing sets for model development and evaluation.

Activity 1.2: Research and Select the Machine Learning Model

The following steps were performed to select the most suitable machine learning algorithm.

- **Understand the Project Requirements:** Analyze the objective of predicting HDI from four socioeconomic indicators.
- **Evaluate Machine Learning Algorithms:** Compare different regression algorithms such as **Linear Regression, Decision Tree Regression, and Random Forest Regression.**
- **Analyze Model Performance:** Evaluate models based on prediction accuracy, simplicity, interpretability, and computational efficiency.
- **Select the Optimal Model:** **Linear Regression** was selected because it provides accurate predictions, is computationally efficient, easy to interpret, and suitable for continuous HDI score prediction.

Activity 1.3: Define the Architecture of the Application

- **Design the System Architecture:** Create an architecture illustrating the interaction between the **user interface, Flask backend, machine learning model, and HDI dataset.**
- **Frontend Functionality:** Users enter **Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita** through a web form.
- **Backend Responsibilities:** The Flask backend validates user inputs, loads the trained **HDI.pkl** model, processes prediction requests, and returns the predicted HDI score and development category.
- **Machine Learning Integration:** The backend sends validated input data to the **Linear Regression** model, receives the predicted HDI score, classifies it into **Low, Medium, High, or Very High**, and displays the results on the web interface.

Activity 1.4: Set Up the Development Environment

- **Install Python and Required Libraries:** Install **Python 3.11** and the required libraries including **NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Flask, and Jupyter Notebook.**
- **Create a Virtual Environment:** Create separate virtual environments for the **Training** and **Flask** modules using Python's **venv** package.

```
D:\projects>python -m venv myenv
```

```
D:\projects>"d:\projects\venv\Scripts\activate"
```

```
(myenv) D:\projects> pip install -r requirements.txt
```

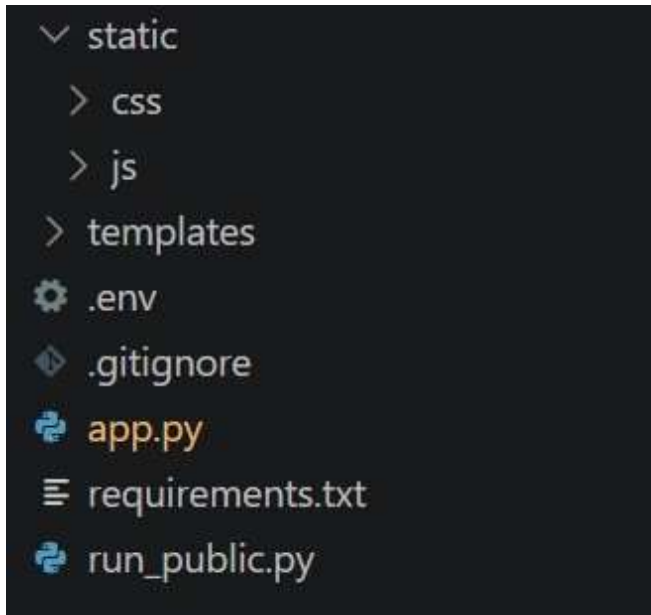
- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a **.env** file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Prediction Features

Implement the core functionalities of the Human Development Index Predictor using Machine Learning.

HDI Score Prediction

- Develop a **Linear Regression** model to predict the Human Development Index (HDI) score using four socioeconomic indicators:
 - Life Expectancy
 - Mean Years of Schooling
 - Expected Years of Schooling
 - Gross National Income (GNI) per Capita

HDI Category Classification

- Classify the predicted HDI score into one of four development categories:
 - Low
 - Medium
 - High
 - Very High

Input Validation

- Validate all user inputs before prediction.
- Ensure numeric values fall within acceptable ranges and display appropriate error messages for invalid inputs.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

Create the following routes in **app.py**:

- / → Home Page
- /**predict** → Prediction Page
- /**about** → About Project Page
- /**api/predict** → REST API for HDI Prediction

Process User Input

- Create a prediction form in **predict.html**.
- Accept user inputs for:
 - Life Expectancy
 - Mean Years of Schooling
 - Expected Years of Schooling
 - GNI per Capita
- Validate the inputs before sending them to the prediction model.

Integrate the Machine Learning Model

- Load the trained **HDI.pkl** model during Flask application startup.
- Convert user inputs into the required format.
- Pass the input values to the Linear Regression model.
- Return the predicted HDI score and corresponding development category to the user interface.

Milestone 3: App.py Development

Milestone 3 focuses on implementing the core application logic in **app.py**, which acts as the central controller of the HDI Predictor. It manages routing, input validation, prediction processing, and communication between the frontend and the machine learning model.

Activity 3.1: Writing the Main Application Logic

Define Flask Routes:

Implement the following routes:

- `/` → Displays the Home page.
- `/predict` → Displays the prediction form and prediction results.
- `/about` → Displays project methodology and HDI information.
- `/api/predict` → Accepts prediction requests and returns JSON responses.

```
@app.route("/")
def home():
    """Home page with project introduction."""
    return render_template("index.html")

@app.route("/predict", methods=["GET", "POST"])
def predict():
    """HDI prediction form page."""
    prediction = None
    category = None
    error = None
    form_data = {}

    if request.method == "POST":
        try:
            life_exp = float(request.form.get("life_expectancy", ""))
            mean_school = float(request.form.get("mean_schooling", ""))
            exp_school = float(request.form.get("expected_schooling", ""))
            gni = float(request.form.get("gni_per_capita", ""))

            form_data = {
                "life_expectancy": life_exp,
                "mean_schooling": mean_school,
                "expected_schooling": exp_school,
                "gni_per_capita": gni,
            }

            error = " ".join(errors)
```

```
@app.route("/api/predict", methods=["POST"])
def api_predict():
    """JSON REST API endpoint for programmatic HDI prediction."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "Invalid or missing JSON body."}), 400

    required_fields = [
        "life_expectancy",
        "mean_schooling",
        "expected_schooling",
        "gni_per_capita",
    ]

    features = pd.DataFrame(
        [[life_exp, mean_school, exp_school, gni]],
        columns=["Life_Expectancy", "Mean_Years_Schooling",
               "Expected_Years_Schooling", "GNI_Per_Capita"],
    )

    raw_pred = float(model.predict(features)[0])
    raw_pred = max(0.0, min(1.0, raw_pred))
    score = round(raw_pred, 3)
```

Implement Route Logic

For the **/predict** route:

- Read user inputs submitted through the HTML form.
- Validate all input values.
- Return an error message if invalid values are entered.
- Load the trained **HDI.pkl** model.
- Generate the HDI prediction.
- Display the HDI score and development category.

Implement Helper Functions

Develop helper functions including:

- **validate_inputs()** – Validates user input values.
- **classify_hdi()** – Converts the predicted HDI score into Low, Medium, High, or Very High.
- **predict_hdi()** – Uses the trained model to generate predictions.

Prediction Flow

- Home Route (**/**) → Displays the application homepage.
- Prediction Route (**/predict**) → Receives user inputs and displays prediction results.
- API Route (**/api/predict**) → Returns JSON containing:

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Create the HTML Structure

Develop the following templates:

- **base.html**
- **index.html**
- **predict.html**
- **about.html**
- **404.html**

Design a Responsive Layout

- Create **style.css** for application styling.
- Build responsive layouts using HTML5 and CSS3.
- Ensure compatibility across desktop and mobile devices.

Create User Interface Components

Develop the following UI sections:

- Home page introducing the HDI Predictor.
- Prediction form for entering socioeconomic indicators.
- Prediction result card displaying:
 - HDI Score
 - Development Category
- About page describing the project and HDI methodology.

Activity 4.2: Display Prediction Results

Handle Form Submission

- Collect user inputs from the prediction form.
- Submit the data to the Flask backend.
- Process the request and return prediction results.

Display Results

- Display the predicted HDI score.
- Show the corresponding development category.
- Display validation messages for incorrect inputs.

Enhance User Experience

- Show prediction results immediately after submission.
- Keep the interface responsive and easy to navigate.
- Ensure smooth interaction between the frontend and backend.

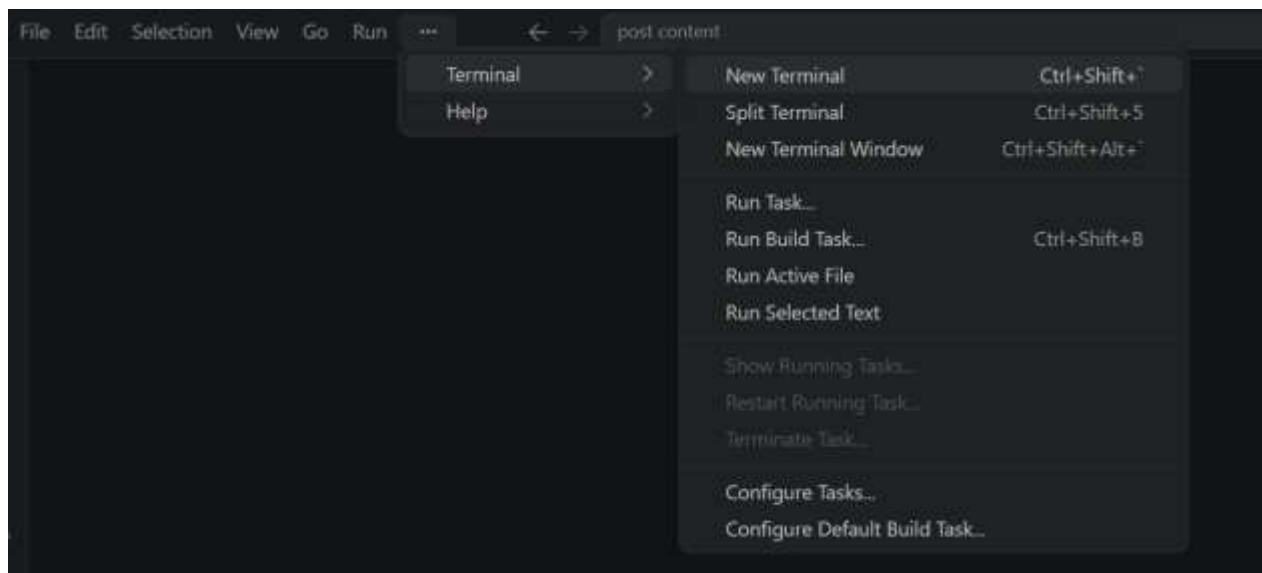
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

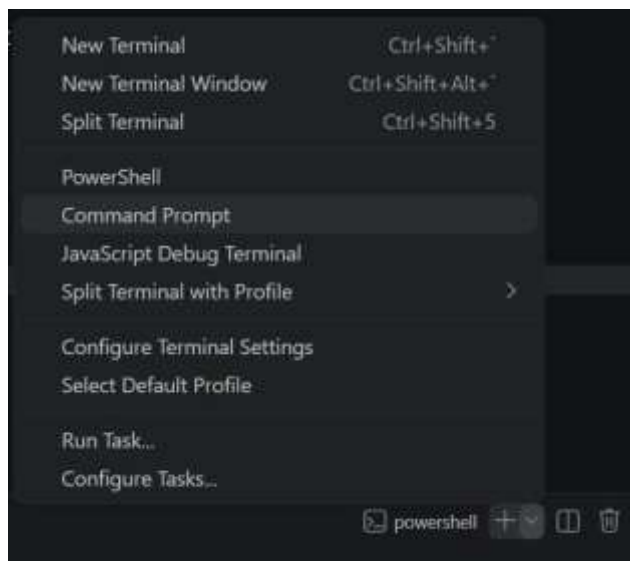
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects>pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment.  
d.  
* Running on http://127.0.0.1:5050  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 442-110-195
```

Activity 5.3: Project Deployment

The HDI Predictor can be deployed to a cloud platform for public access if required.

Deployment Options

- Deploy the Flask application using platforms such as **Render**, **Railway**, or **PythonAnywhere**.
- Configure the project dependencies using the `requirements.txt` file.
- Upload the trained **HDI.pkl** model and the application source code.
- Start the Flask application on the hosting platform and verify that the prediction system functions correctly.

Deployment Verification

- Access the deployed application using the generated public URL.
- Verify that:
 - The Home page loads successfully.
 - The Prediction page accepts valid inputs.
 - The HDI score and development category are displayed correctly.
 - Invalid inputs are handled gracefully.

Run the Public Deployment Script:

- Instead of running `app.py` directly, launch `run_public.py` to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python app.py
```

```
===== *
Serving Flask app 'app'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5000/
Press CTRL+C to quit

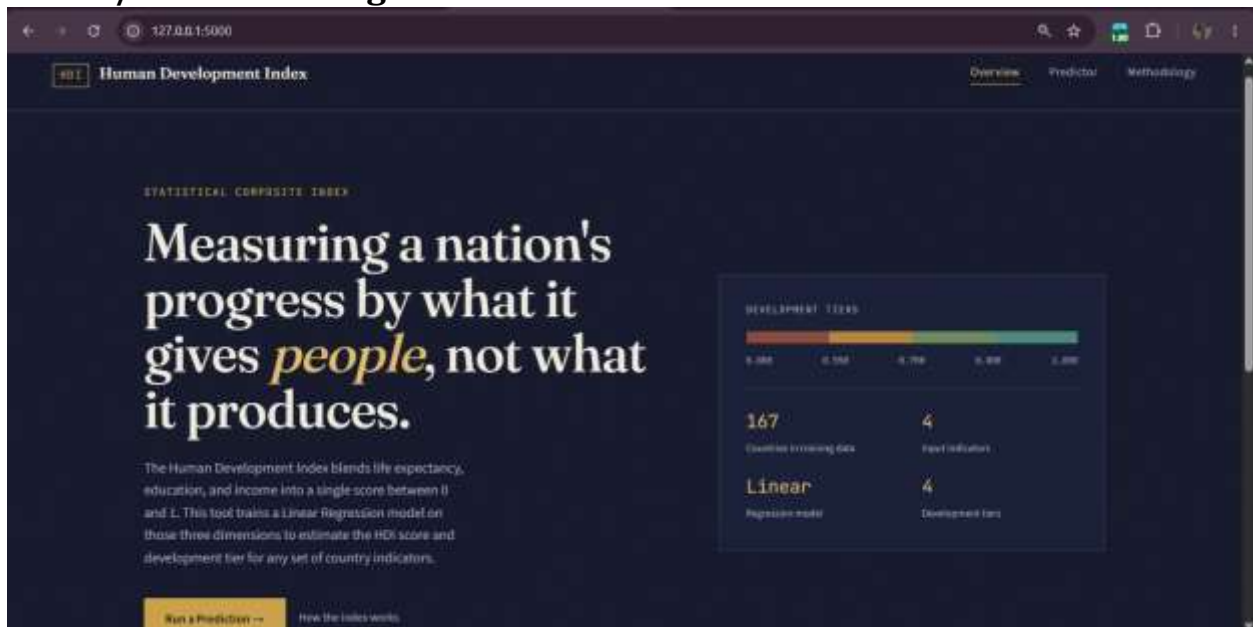
* Restarting with stat
* Debugger is active!
* Debugger PIN: -----
=====
```

Important Notes:

- Ensure the virtual environment is activated before running the application locally.
- Keep the trained **HDL.pkl** model in the **Flask** folder so it can be loaded during application startup.
- Verify that all required Python libraries are installed using `requirements.txt`.
- If deployed online, ensure the application and model files are correctly configured on the hosting platform.

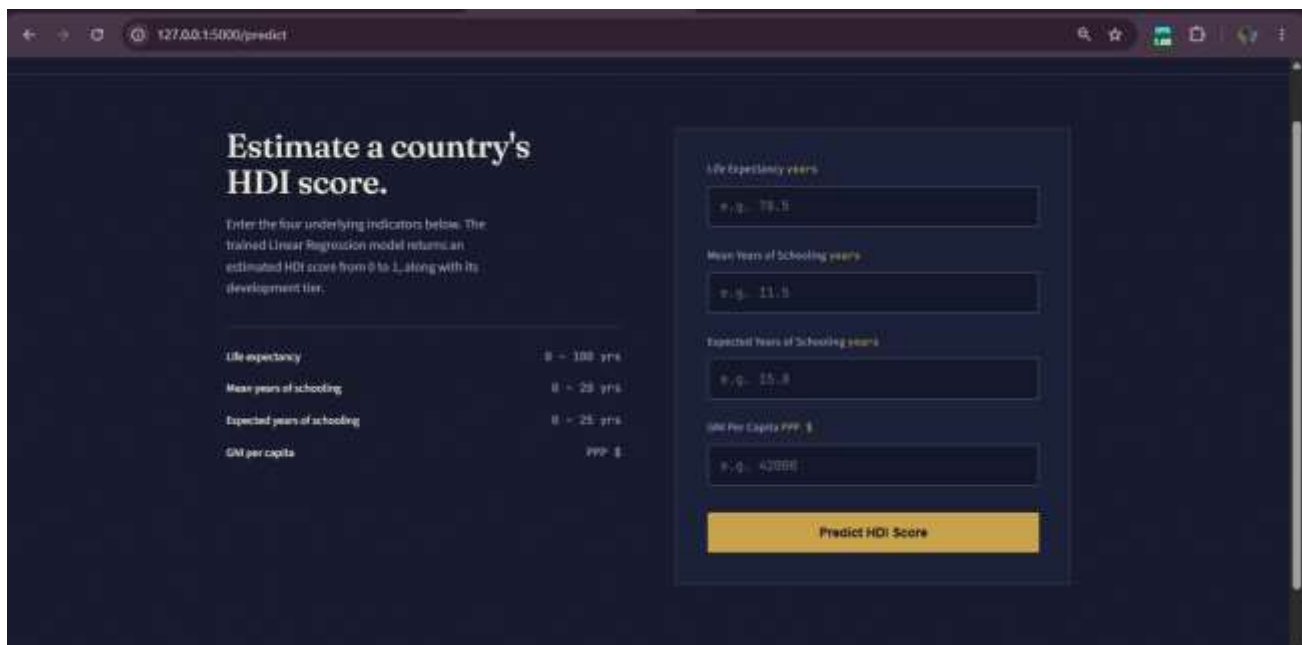
Exploring the Website's Web Pages:

Home / Generator Page:



Description: The web interface of the **Human Development Index (HDI) Predictor** serves as both the landing page and the prediction platform. It features a modern, responsive design with a clean navigation bar, informative overview section, and visually appealing statistical cards. The application includes dedicated pages for **Overview**, **Predictor**, and **Methodology**, where users can enter **Life Expectancy**, **Mean Years of Schooling**, **Expected Years of Schooling**, and **GNI per Capita** to instantly receive the predicted **HDI score** and corresponding **Human Development category (Low, Medium, High, or Very High)** using a trained **Linear Regression** model.

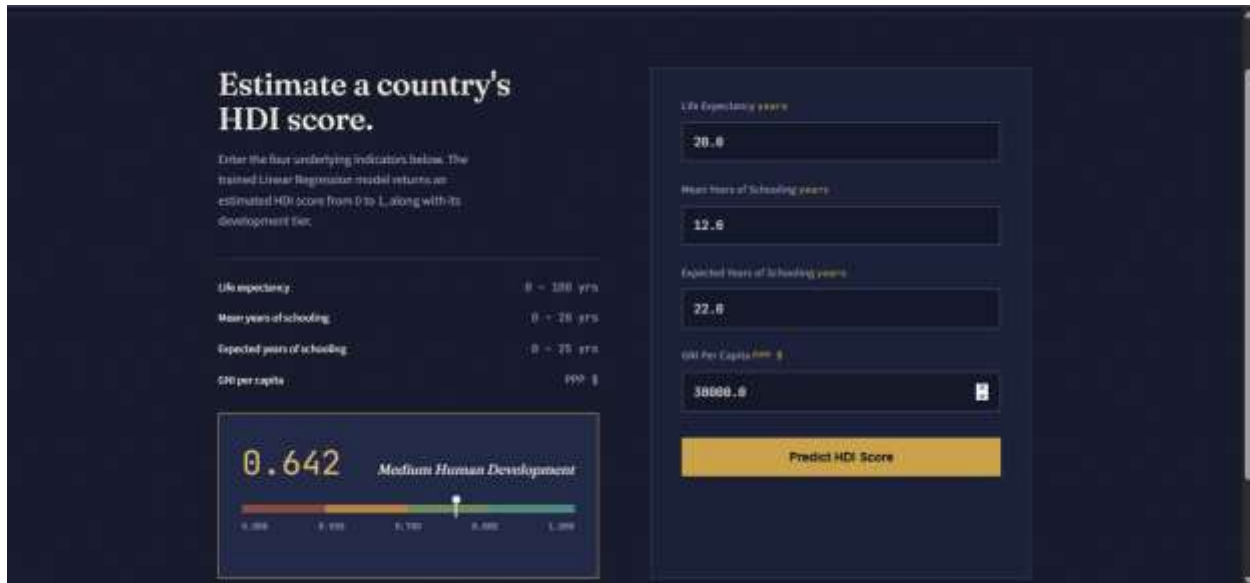
Input Section:



The screenshot shows the input section of the Human Development Index (HDI) Predictor. The page has a dark blue background with white and yellow text. The main heading reads "Estimate a country's HDI score." Below this, a paragraph explains that the user should enter four underlying indicators: Life expectancy, Mean years of schooling, Expected years of schooling, and GNI per capita. To the right, there are four input fields with placeholder text: "Life expectancy years" (e.g., 75.5), "Mean years of schooling years" (e.g., 11.5), "Expected years of schooling years" (e.g., 15.8), and "GNI Per Capita PPP, \$" (e.g., 42000). At the bottom, there is a "Predict HDI Score" button.

Description: A modern, responsive prediction form allows users to enter the four key Human Development Index indicators: **Life Expectancy**, **Mean Years of Schooling**, **Expected Years of Schooling**, and **Gross National Income (GNI) per Capita**. Input guidelines and valid ranges are displayed alongside the form to assist users in entering accurate values. After clicking the **"Predict HDI Score"** button, the Flask application validates the inputs, processes them using the trained **Linear Regression** model, and returns the predicted **HDI score** along with its corresponding development category (**Low, Medium, High, or Very High**).

Results Section:



The screenshot displays a web application interface for estimating a country's HDI score. The main heading is "Estimate a country's HDI score." Below it, a sub-heading states: "Enter the four underlying indicators below. The trained Linear Regression model returns an estimated HDI score from 0 to 1, along with its development tier."

On the left, there are four input fields with their respective ranges:

- Life expectancy: 0 - 100 yrs
- Mean years of schooling: 0 - 20 yrs
- Expected years of schooling: 0 - 20 yrs
- GNI per capita: 0 - 100,000 \$

Below these fields is a color-coded scale for the HDI score, ranging from 0.000 to 1.000. The scale is divided into four tiers: Low (red), Medium (orange), High (green), and Very High (blue). The predicted score of 0.642 is shown in the center of the scale, corresponding to the "Medium Human Development" category.

On the right, there are four input fields for the indicators, each with a value entered:

- Life expectancy years: 20.0
- Mean years of schooling years: 12.0
- Expected years of schooling years: 22.0
- GNI Per Capita (USD): 30000.0

Below these fields is a yellow button labeled "Predict HDI Score".

Description: After the user clicks the **"Predict HDI Score"** button, the prediction results are displayed dynamically on the same page. The results panel presents the **predicted HDI score** along with its corresponding **Human Development category (Low, Medium, High, or Very High)**. A color-coded HDI scale visually indicates the predicted score's position across the four development tiers, helping users easily interpret the country's development status. The prediction form remains available, allowing users to modify the input values and instantly generate new predictions without leaving the page.

Conclusion:

A Comprehensive Measure of Well-Being is a machine learning-based web application developed using **Python, Flask, and Scikit-learn** to predict a country's **Human Development Index (HDI)**. By utilizing a **Linear Regression** model trained on the Human Development Index dataset, the application enables users to estimate the **HDI score** and classify it into **Low, Medium, High, or Very High** human development categories based on four key socioeconomic indicators. The project demonstrates the complete machine learning workflow, including data preprocessing, exploratory data analysis, model training, evaluation, model serialization, and deployment through a user-friendly Flask web application.

Looking ahead, **A Comprehensive Measure of Well-Being** can be enhanced by incorporating advanced machine learning algorithms for improved prediction accuracy, integrating real-time socioeconomic datasets, providing interactive visualizations and country-wise comparisons, and deploying the application on cloud platforms for wider accessibility. These enhancements would transform the system into a comprehensive decision-support tool for students, researchers, policymakers, and development organizations, enabling more informed analysis of global human development.